```
    at App (<anonymous>)
```

The `componentDidCatch` method receives both the `error` and `errorInfo` objects, which enables additional error processing or logging. Listing A.5 also demonstrates how to leverage the error information. The console output of `error` is shown in listing A.7.

Listing A.7 Console output of error for an uncaught error logged by ErrorBoundary component

```
ErrorBoundary.tsx:28 Error caught by ErrorBoundary: Error: This i
    at BuggyComponent (ErrorBoundaryExample.tsx:4:9)
 Error Component Stack
    at ErrorBoundary (ErrorBoundary.tsx:17:5)
    at ErrorBoundaryExample (<anonymous>)
    at App (<anonymous>)
```

The console output of `errorInfo.componentStack` is shown in listing A.8.

Listing A.8 Console output of errorInfo.componentStack for an uncaught error logged by ErrorBoundary component

```
Error details:
    at BuggyComponent (http://localhost:5173/src/use-cases/ErrorB
    at ErrorBoundary (http://localhost:5173/src/components/error-
    at ErrorBoundaryExample (<anonymous>)
    at App (<anonymous>)
```

The `render` method can also be used to handle special error scenarios. For instance, the `render` method in listing A.9 selectively displays an error message, effectively ignoring errors originating from:

- *New Relic's JavaScript Agent*—New Relic is a monitoring platform that provides insights into application performance. Its JavaScript agent is a small script that collects data about the application's performance, errors, and user interactions. In this context, the `render` method ignores errors related to New Relic's data collection.
- *Ant Design's* `parentElement` *property*—Ant Design is a popular UI component library. The `parentElement` property is likely a reference to a parent DOM element in the component hierarchy. The `render` method ignores errors related to accessing or manipulating this property.